

Activity guide — Exploring external tables (DIN §30): Bronze made tangible

Day 2 (Tue Oct 6), 11:15–11:35, instructor-led follow-along. You will point BigQuery at a bucket of Hive-partitioned Parquet files and query them as a table — with no load job, no SSIS package, and no staging table. Adapted from §30 of the [Do-It-Now \(DIN\) activities page](#).

What you'll do

1. Inspect a public GCS bucket (gs://jwd-gcp-demos/orders_partitioned/) laid out in Hive partition style.
2. Create a BigQuery **external table** over it, named `ext_part`, in your class dataset.
3. Run three small queries and watch **bytes processed**: a full scan, a filter on a data column (still a full scan), and a filter on the partition column (bytes drop dramatically).

Cost: effectively free — the DIN page badges this activity low-cost, and the scanned bytes on this dataset are tiny (three small queries; pennies at on-demand rates).

Why it matters for your migration

This is Bronze made tangible. Open-format files sit in a bucket, partitioned by date, and are immediately queryable in standard SQL — zero loading. Your architecture doc's Bronze layer is exactly this.

- Today, querying a county file drop means an SSIS package and a staging table first. Here **the file is the table**.
- For the M&O team: there is no database to patch and no service to keep alive. Operating Bronze is bucket IAM and lifecycle policies, not servers.
- The honest trade-off: external tables get **no BigQuery storage optimizations** — no native clustering, slower repeated scans, and the source of truth lives outside BigQuery. That is precisely why your Silver and Gold become *managed* tables built by Dataform. Bronze is for landing and exploring; Silver/Gold are for serving.

Before you start

- You need the **class dataset** you created on Day 1 morning (DIN §3). If you don't have it: in the BigQuery **Explorer** pane, click your project, then the three-dot menu → **Create dataset** → Dataset ID class → **Create dataset**.
- Open the **BigQuery console**.
- Open the SQL file for this activity — the local copy in this folder, `external_hive_example.sql` (byte-identical to the [course repo's copy](#); the DIN page links the same file).
- In another browser tab, open the bucket browser:
https://console.cloud.google.com/storage/browser/jwd-gcp-demos/orders_partitioned

Step 1 — Look at the bucket first

Browse `gs://jwd-gcp-demos/orders_partitioned/` before creating anything, and answer for yourself:

- How many "directories" are there in the bucket?
- How many files does each directory hold?
- What does the naming of the directories imply?

What you are seeing is a **Hive-partitioned dataset living in a data lake on GCS**: directories named `order_date=YYYY-MM-DD/` — the Hive key=value partition layout — each holding Parquet file(s). Spotting that naming convention yourself is the payoff: the *path* is metadata.

Step 2 — Create the external table

1. In the BigQuery **Explorer** pane, select your class dataset.
2. From its three-dot menu, select **Create table**.
3. In **Create table from**, select **Google Cloud Storage**.
4. In **Select file from GCS bucket or use a URI pattern**, enter `jwd-gcp-demos/orders_partitioned/*`
5. In **File format**, select **Parquet**.
6. Check **Source data partitioning**. (Leave the partition inference mode at its default, automatically infer types.)
7. In **Select Source URI Prefix**, enter `gs://jwd-gcp-demos/orders_partitioned/`
8. Under Destination: in **Table**, enter `ext_part`; set **Table type** to **External table**.
9. In **Schema**, select **Auto detect**.
10. Click **CREATE TABLE**.
11. Open the new table's details. Notice what is *not* there: no storage size of its own. The table is a metadata pointer at `gs://jwd-gcp-demos/orders_partitioned/` — delete the table later and every file in the bucket is untouched.

Step 3 — Query it, and watch the bytes

Run the three queries from `external_hive_example.sql` one at a time. After each run, open the job's **Execution details** and note how much data was processed.

1. **Query the external table** — `SELECT * FROM class.ext_part` Full scan: BigQuery opens every Parquet file in every partition folder. Rows returned = the whole dataset.
2. **Query with a WHERE on a data column** — `WHERE order_num="68 610 383-54"` Still scans (almost) everything. `order_num` is a *data* column, not the partition key, so BigQuery must open every file to find that order. A WHERE clause does not automatically save you money — say that sentence to your neighbor.
3. **Query on the partition column** — `WHERE order_date="2018-01-01"` Bytes processed drops dramatically. Because you checked **Source data partitioning**, BigQuery

inferred `order_date` from the folder names and reads **only the matching partition's files**.

That third query is the lesson: **partition pruning on raw files in a bucket — and no load job ever ran.**

Common gotchas

- **The two URI fields want different formats.** The file pattern is `jwd-gcp-demos/orders_partitioned/*` — with the `*` wildcard, no `gs://`. The Source URI Prefix is `gs://jwd-gcp-demos/orders_partitioned/` — with `gs://`, no wildcard. Swap or blur them and table creation fails.
- **Don't forget the `*`.** Without the wildcard, BigQuery can't match the files inside the partition folders.
- **Don't skip the 'Source data partitioning' checkbox.** Without it, `order_date` never becomes a column — the third query errors on an unrecognized name instead of pruning partitions.
- **Schema: use Auto detect, and relax.** Parquet files carry their schema inside the file, so autodetect is safe here — the CSV autodetect surprises you saw on Day 1 don't apply to Parquet.
- **Table type must be External table.** Otherwise BigQuery tries to *load* the data into managed storage instead of pointing at it — the opposite of what this activity is about.
- **The SQL assumes your dataset is literally named `class`.** If yours is named differently, adjust `class.ext_part` in the queries.

Self-check

- ☐ `ext_part` exists in my `class` dataset, and its details show an external source URI — not stored bytes.
- ☐ Query 1 returned the full dataset (all partitions, every file).
- ☐ Query 2 (`order_num`) scanned roughly the same bytes as query 1 — and I can explain why.
- ☐ Query 3 (`order_date`) processed dramatically fewer bytes — and I can explain why.
- ☐ I can say why this is "Bronze," and what would turn this data into Silver (a managed, cleaned BigQuery table — built by Dataform).

If you finish early

- Pick another `order_date` folder you saw in the bucket browser and run the partition query for that date; compare bytes processed.
- Run `SELECT order_date, COUNT(*) FROM class.ext_part GROUP BY order_date` and predict the bytes before you run it — why is it a full scan?
- Compare the bytes processed of query 2 and query 3 side by side in your job history. That ratio is the argument you'll make in your own design reviews for partitioning Bronze by date.

- Reopen the table details and study the external data configuration — this is the definition you'd script later with bq mkdef or DDL.

What this replaces in your current stack

- **The SSIS package + staging table** you build today before anyone can query a county or vendor file drop. Bronze is queryable the moment files land.
- **The 'copy it into the warehouse before it's useful' reflex.** Copy when you have a reason (performance, governance, conformance) — that's Silver/Gold — not as a reflex.
- A preview of this afternoon: Iceberg upgrades exactly this pattern — what if the files in the bucket could also get ACID semantics and schema evolution? Keep ext_part in mind.

Dig deeper

- [Create Cloud Storage external tables](#) — the console fields you just used, plus the DDL/bq equivalents
 - [Manage external Hive partitioned data](#) — partition layouts, inference modes, pruning rules
 - [Introduction to BigLake tables](#) — the governed upgrade path for external tables
 - The [Do-It-Now \(DIN\) activities page](#) — §30 is this activity; §3 created your class dataset
-

Part of the Iowa DOM DoIT Data Engineering student pack — see ../00_schedule.md.